

Resumo Completo de Engenharia de Software e DevOps

Baseado nos Slides da Cadeira

ES 01: Princípios de Engenharia de Software e Agile

Foco na transição de gestão tradicional para ágil e frameworks de trabalho.

- **Ciclo de Vida (SDLC):** O processo clássico envolve Análise, Design, Construção, Testes e Entrega.
 - **Waterfall (Cascata) vs. Agile:**
 - **Waterfall:** Modelo "Plan-driven". As fases são sequenciais. Fácil de planejar, mas assume que os requisitos são fixos. Risco de falha tardia ("Big Bang").
 - **Agile:** Focado em entregar valor incrementalmente. Aceita a mudança.
 - **Gestão de Projetos:**
 - Os 3 pilares: **Planear** (definir scope/recursos), **Executar** (coordenar tarefas), **Entregar** (medir sucesso).
 - **Scrum (Framework Ágil):**
 - **Papéis:**
 - *Product Owner (PO):* Define o "quê". Gere o Product Backlog e maximiza o valor para o cliente.
 - *Scrum Master (SM):* Garante o processo. Remove impedimentos. Líder servidor.
 - *Development Team:* Auto-organizada, multifuncional, constrói o incremento.
 - **Artefactos:** Product Backlog (lista viva de requisitos), Sprint Backlog (tarefas da iteração), Incremento (software funcional).
 - **Cerimónias:**
 - *Sprint Planning:* O que vamos fazer e como?
 - *Daily Scrum:* Sincronização diária (não é reporte a gestão).
 - *Review:* Demo ao cliente.
 - *Retrospective:* Melhoria contínua do processo da equipa.
-

ES 02: Planeamento de Requisitos no Ciclo DevOps

Como definir, partir e estimar trabalho de forma eficiente.

- **User Stories:**
 - Formato padrão: "Como <tipo de utilizador>, eu quero <funcionalidade> para que <razão/valor>".
 - Devem seguir o critério **INVEST**: Independentes, Negociáveis, Valiosas, Estimáveis, Small (Pequenas), Testáveis.
 - **Requisitos:**
 - *Funcionais (FR)*: O que o sistema faz (ex: "O sistema deve permitir login").
 - *Não-Funcionais (NFR)*: Como o sistema se comporta (ex: Segurança, Performance < 500ms, Usabilidade).
 - **Estratégias de Fatiamento (Slicing):**
 - **Vertical Slicing (Correto)**: Cortar uma funcionalidade através de todas as camadas (Frontend + Backend + DB) para entregar algo pequeno mas funcional.
 - **Horizontal Slicing (Evitar)**: Fazer "toda a base de dados" primeiro. Não entrega valor visível ao utilizador até ao fim.
 - **Estimativas:**
 - Uso de **Story Points** (sequência de Fibonacci: 1, 2, 3, 5, 8...) para medir complexidade e esforço relativo, em vez de horas absolutas.
 - Técnica de **Planning Poker** para obter consenso na equipa.
-

ES 03: Componentes de Software

Arquitetura e organização técnica do projeto.

- **Mapeamento Requisitos -> Arquitetura:**
 - Cada User Story deve mapear para componentes de software. Stories complexas podem exigir coordenação entre vários serviços.
 - Rastreabilidade (Traceability): Garantir que o código pode ser ligado a uma necessidade do utilizador.
- **Refactoring:**
 - Mudanças na estrutura interna sem alterar comportamento externo.
 - Evitar "Big Bang deployments" (grandes mudanças de uma vez). Preferir pequenas alterações incrementais.
- **Tecnologias do Projeto (Exemplo Prático):**
 - Backend: Spring Boot.

- Frontend: ReactJS.
 - Comunicação: RESTful API (Front-Back) e Apache Kafka (entre serviços/Message Queue).
 - Identity Provider: Keycloak.
 - **Guia Prático:** Implementar user stories usando "Pull Requests" separadas, focadas em fatias verticais.
-

ES 04: Continuous Deployment (Automação)

Automatizar o caminho do código até à produção.

- **O problema "Dev vs Ops":** A "Wall of Confusion" onde Devs querem mudança (features) e Ops querem estabilidade. DevOps quebra esta barreira.
 - **Conceitos:**
 - **CI (Continuous Integration):** Merge frequente de código (várias vezes ao dia) com testes automáticos.
 - **CD (Continuous Deployment):** Qualquer mudança que passa nos testes vai automaticamente para produção.
 - **GitHub Actions (Ferramenta de CI/CD):**
 - **Workflows:** Processos automatizados definidos em YAML (.github/workflows).
 - **Triggers:** Eventos que iniciam o pipeline (ex: push, pull_request, schedule).
 - **Runners:** Servidores que executam os jobs (podem ser *GitHub-hosted* ou *Self-hosted*).
 - **Actions:** Blocos reutilizáveis de código (ex: actions/checkout, actions/setup-java). Podem ser Containers (Docker) ou JavaScript.
 - **Lab:** Criar pipelines para Build (com Docker multi-stage) e Deploy.
-

ES 05: Continuous Delivery (Estratégias de Entrega)

Como lançar software de forma segura e controlada.

- **Release Candidate:** Uma versão que *pode* ir para produção, sujeita a validação final.
- **Princípios:** Processo repetível e fiável. Automatizar tudo. Manter tudo em controlo de versão.
- **Estratégias de Deployment:**
 - **Blue/Green:**

- Dois ambientes (Blue=Live, Green=Novo).
 - Deploy é feito no Green. Testes correm lá.
 - Troca de tráfego instantânea (Router/Load Balancer).
 - *Vantagem*: Rollback imediato. *Desvantagem*: Custo (dobro da infraestrutura).
 - *Técnica UNIX*: Links simbólicos para pastas de versões diferentes.
 - **Canary**: Lançar para um subconjunto de utilizadores primeiro.
 - **Feature Toggles (Flags)**:
 - Permite separar o *Deployment* (código nos servidores) da *Release* (funcionalidade visível ao utilizador).
 - Ferramenta mencionada: **Flagsmith**.
 - **Bases de Dados**: O maior desafio. Usar padrão "Expand and Contract" para manter retrocompatibilidade durante migrações.
-

ES 06: Testes Avançados

Garantia de qualidade e tipos de testes.

- **A história do Mars Climate Orbiter**: Exemplo de falha catastrófica devido a falta de testes de integração (erro de conversão de unidades Métricas vs Imperiais).
 - **Pirâmide de Testes**:
 1. **Unitários (Base)**: Rápidos, baratos, isolados. A maior quantidade.
 2. **Integração**: Verificam comunicação entre módulos/serviços.
 3. **E2E (Topo)**: Ponta-a-ponta, simulam o utilizador. Lentos, frágeis e caros.
 - **Anti-Padrão "Ice Cream Cone"**: Muitos testes manuais/UI e poucos unitários. Invertido e ineficiente.
 - **Integração CI**:
 1. Os testes devem correr a cada commit.
 2. Falhar o pipeline se os testes falharem ("Break the build").
 3. Ferramenta de report: **Allure Reports**.
-

ES 07: Infrastructure as Code (IaC)

Gerir infraestrutura como software.

- **Evolução**:
 - *Pets (Animais de estimação)*: Servidores manuais, com nomes, cuidados individualmente. Se morrem é crise.

- *Cattle (Gado)*: Servidores automatizados, números em vez de nomes, descartáveis e substituíveis.
 - **CAPEX vs OPEX**: Mudança de investimento capital (hardware físico) para despesa operacional (Cloud/Pay-as-you-go).
 - **Terraform**:
 - Ferramenta **Declarativa** (dizemos "o que queremos", não "como fazer").
 - **Estado (tfstate)**: Ficheiro crítico que mapeia o mundo real à configuração.
 - **Providers**: Plugins para interagir com APIs (AWS, Docker, Azure).
 - **Alta Disponibilidade (HA)**:
 - SLA de 99.9% = ~8h downtime/ano.
 - SLA de 99.99% = ~52 min downtime/ano.
 - Redundância remove pontos únicos de falha (SPOF).
 - *Lab*: Usar Terraform para lançar infraestrutura com Nginx como Load Balancer e réplicas de backend/frontend.
-

ES 08: Monitoring & Observability

Operação e saúde do sistema em produção.

- **SLA / SLO / SLI**:
 - **SLA (Agreement)**: Contrato com o cliente (tem penalidades financeiras).
 - **SLO (Objective)**: Objetivo interno de fiabilidade (ex: 99.9%).
 - **SLI (Indicator)**: A métrica real medida (ex: latência atual é 200ms).
 - **Error Budget**: A margem de erro permitida (100% - SLO). Se esgotar, para-se os deploys de novas features.
- **Monitorização vs. Observabilidade**:
 - *Monitorização*: Diz "Quando" e "O quê" falhou (Dashboards conhecidos).
 - *Observabilidade*: Diz "Porquê" falhou (permite explorar "unknown unknowns").
- **Os 3 Pilares**:
 - **Logs**: Eventos detalhados (ex: `NullPointerException`). Estruturados são melhores.
 - **Métricas**: Dados agregados (ex: uso de CPU, pedidos/seg). Baratos de armazenar.
 - **Tracing (Rastreo Distribuído)**: Segue um pedido através de microsserviços para encontrar gargalos.
- **Ferramentas**: OpenTelemetry (standard), Elastic Stack (Kibana/Elasticsearch).



The image depicts the software development lifecycle (SDLC), which is a structured process used to create and maintain software. It outlines a sequence of phases that guide developers from the initial idea to the final product and its ongoing support. This cyclical model ensures that software is well-planned, designed, and implemented to meet user needs.

The first phase is '**Plan**,' where the project's scope, objectives, and resources are defined. This stage involves gathering requirements from stakeholders, conducting feasibility studies, and creating a detailed project plan. A well-thought-out plan sets the foundation for the entire development process.

Following the 'Plan' is the '**Analyze**' phase, where the requirements are thoroughly examined and documented. This involves creating functional and technical specifications that detail how the software will work. The goal is to ensure a clear understanding of what needs to be built before moving into the design phase.

Next is the '**Design**' phase, where the system's architecture, user interface, and data structures are created. This phase translates the requirements into a detailed design that can be used by the development team. Prototypes and mockups are often created to visualize the final product and gather feedback.

The '**Develop**' phase is where the actual coding and programming take place. Developers write the source code based on the design specifications. This phase also includes unit testing to ensure that individual components of the software work correctly.

After development, the '**Implement**' phase involves deploying the software to a live environment. This includes installation, configuration, and user training. The goal is to make the software available to its intended users and ensure a smooth transition.

The final phase is '**Maintain**,' where the software is supported and updated after it has been deployed. This includes fixing bugs, adding new features, and ensuring the system remains stable and secure. The cyclical nature of the SDLC means that the process can be repeated as new requirements arise, leading to continuous improvement.

